

一、思考题

1、你有过多人协同开发的经历吗？如果有，你们是用什么方式分工协作的？

没有。

2、Git 为什么要设计「暂存(add)-提交(commit)」两个步骤？谈谈你的思考。

(1) 可以灵活筛选提交内容：不用一次性提交所有修改，方便按逻辑分批提交，

(2) 方便缓冲检查：可以确认要提交的文件和改动，避免误提交临时文件。

(3) 拆分工作步骤：把选中要改动的地方和记录改动的版本区分开，提高版本提交的严谨性

3.git branch 和 git branch-a 两条命令的区别是什么？

git branch 只显示本地的所有分支和当前所在的分支。而 git branch-a: 显示所有分支，包括本地分支和远程仓库的分支。、

二、文档阅读与概括

1、Commit Message 规范

介绍了业界广泛采用的 Angular 风格提交信息规范，其核心目的是让 Git 提交信息结构化、标准化，从而提升版本历史的可读性。格式化具备三方面核心作用：能够让开发者快速浏览提交的核心目的，支持按类别过滤提交记录，还可以基于规范的提交历史自动生成项目更新日志。

标准的提交信息由 Header、Body、Footer 三部分构成，其中 Header 必填项，遵循<type>(<scope>): <subject>的固定格式，通过 feat、fix、docs、style、refactor、test、chore 七种类型标识提交类别，搭配可选的范围说明与简短的主题描述；Body 与 Footer 为可选项，分别用于补充详细的变动说明、标注破坏性变更或关联关闭 Issue。

Commitizen、validate-commit-msg、conventional-changelog 等配套工具，可以分别实现交互式生成规范提交、自动校验提交格式、自动生成更新日志，进一步提升落地效率。

2、Git Flow 分支控制

介绍了 Git Flow 分支管理工作流，它通过定义职责清晰的分支体系与严格的合并规则，系统性解决多人协作开发中的代码版本混乱问题。

Git Flow 将分支分为永久分支与临时分支两类：永久分支包含对应线上生产环境稳定代码的 master 分支，以及作为日常开发集成分支的 develop 分支，后者基于前者创建；临时分支则包含用于开发单个新功能的 feature 分支、用于版本发布前测试与准备的 release 分支，以及用于紧急修复线上故障的 hotfix 分支，三类临时分支完成对应任务后合并回来源分支并删除。

这套分支体系既支持多人并行开发不同功能，又能通过规范的合并流程保证主干代码的稳定性，既可以通过原生 Git 命令手动实现，也可以通过 git-flow 扩展工具简化操作。

3、为什么要学习 Git

(1) Git 的版本回溯能力让代码修改拥有可控的历史记录，同时规范的提交习惯能让项目历史清晰可追溯，极大提升后续维护与版本迭代的效率。

(2) 不同开发者可以在独立分支上开发不同功能，通过合并操作高效整合代码，避免了传统文件传输方式的版本覆盖与代码混乱问题，而统一的 Commit Message 规范则让团队提交历史标准化，大幅降低团队沟通与协作成本

三、实验步骤与必要截图

1、装好 Git，生成 SSH 密钥，绑定到 GitHub 上

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ ssh -T git@github.com
Hi Liora-libra! You've successfully authenticated, but GitHub does not provide shell access.
```

2、把课程的模板仓库克隆到本地电脑里。

3、修改 main.c 里的代码，用 make 编译运行后提交代码，推送到 GitHub 仓库。

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ cd ~/ICS2026/ics-yangxinyuan
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ ./main
My name is Yang Xinyuan.
```

4、新建一个 feature 分支，切换到这个分支里，把 main.c 的同一行代码改成别的内容，提交保存。

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ git switch -c feature
Switched to a new branch 'feature'
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ nano main.c
```

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ cat > main.c <<'EOF'
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("My age is nineteen\n");
}
EOF
```

5、切回 main 主分支，把 main.c 的同一行改成和 feature 分支不一样的内容，提交保存。

6、在 main 分支里合并 feature 分支，触发合并冲突，然后手动把冲突改好，提交解决后的结果。

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ git log --oneline
8401416 (HEAD -> main, origin/main, origin/HEAD) resolve merge conflict between main and feature
3c7ba8b main: keep original print message
a61df64 (feature) feature: update print message
09ed232 remove compiled binary files
813b328 lab1: print my name
d68d37e Initial commit
```

```
yangxinyuan@localhost:~/ICS2026/ics-yangxinyuan$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("My name is Yang Xinyuan.\n");
}
```

